



MIDDLE EAST TECHNICAL UNIVERSITY

DEPARTMENT OF MECHANICAL ENGINEERING

Tuned Mass Dampers
14th Summer Research Project

Video Link: [Summer Project Demonstration - YouTube](#)

Muhammed Ömer
YİĞİT

Ege Sarp
DENGİZMEN

Table of Contents

1 Introduction	3
2 Tuned Mass Dampers	4
2.1 Two Degrees of Freedom System with a Harmonic Disturbance.....	4
2.1.1 Optimization for 2-DOF System	5
H2 Optimization	6
H_{∞} Optimization	7
2.2 Three Degree of Freedom Systems with a Harmonic Disturbance	9
2.2.1 3-DOF Systems with Parallel Connection	9
2.2.1.1 Optimization	10
H2 Optimization	10
H_{∞} Optimization	12
2.2.2 3-DOF System with Series Connection	13
2.2.2.1 Optimization	13
H2 Optimization	13
H2 Optimization	14
Additional Discussion.....	15
3 Demo Project	16
3.1 Physical Model.....	16
3.2 Analysis of Base Structure	17
3.3 Construction and Optimization of TMD.....	20
3.4 Results and Conclusion.....	21
4 Appendix.....	22
4.1 MATLAB Code for Optimization	22
4.2 Arduino Code for Operation	27
4.3 Arduino Code for Data Collection.....	29
4.4 Python Code for Logarithmic Decrement.....	30
4.5 MATHCAD Screen Shots	33

1 Introduction

Prof. Dr. Ender Ciğeroğlu from the Department of Mechanical Engineering at Middle East Technical University invites his students, who could pass the course ME302 Theory of Machines II, to a summer research project on mechanical vibrations. In ME302 Theory of Machines II, students are taught the virtual work method, fundamentals of single degree of freedom mechanical vibrations, and free undamped two degrees of freedom systems. Students have been invited to the summer research projects since 2010. In summer research projects, students have a great chance to improve their theoretical, academical, and computer skills. The project mainly focuses on two and three degrees of freedom tuned mass dampers.

The 14th Summer Research Project was held in 2023. In this paper, applications of 14th Summer Research Project which consist of solution for tuned mass damper systems with varying degrees of freedom, H_2 and H_∞ optimization for these systems using MATLAB and MATHCAD, and a simple two degrees of freedom demo project.

2 Tuned Mass Dampers

Tuned mass dampers are used to absorb the mechanical vibrations of a host structure. The host structure generally undergoes a harmonic disturbance in the form of $F \sin(\omega t)$. The first figure that is modelled and solved is two degrees of freedom system, the second figure is three degrees of freedom with parallel connection system, and the third figure is three degrees of freedom with series connection system.

2.1 Two Degrees of Freedom System with a Harmonic Disturbance

This system consists of a host structure with the parameters of $m_1 (kg)$ standing for mass of the structure, $k_1 (N/m)$ standing for the stiffness of the structure, $c_1 (Ns/m)$ standing for the damping coefficient of the structure. An example of the system is shown in **Figure 1**. Equations of motion for this system can be expressed as:

$$M\ddot{x} + C\dot{x} + Kx = F,$$

M, C , and K are matrices which can be shown as:

$$M = \begin{bmatrix} m_1 & 0 \\ 0 & m_2 \end{bmatrix}, C = \begin{bmatrix} c_1 + c_2 & -c_2 \\ -c_2 & c_2 \end{bmatrix}, K = \begin{bmatrix} k_1 + k_2 & -k_2 \\ -k_2 & k_2 \end{bmatrix}$$

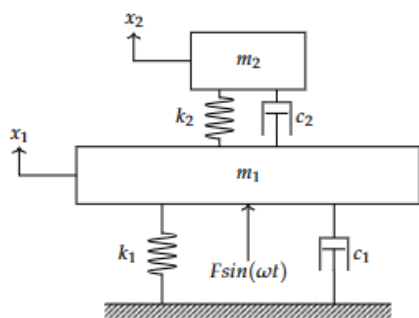


Figure 1: 2-DOF System with a harmonic disturbance on the host structure.

Also, $\ddot{x}, \dot{x}, x, F$ are vectors which can be shown as:

$$\ddot{x} = \begin{pmatrix} \ddot{x}_1 \\ \ddot{x}_2 \end{pmatrix}, \dot{x} = \begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \end{pmatrix}, x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, F = \begin{pmatrix} F_1 \\ F_2 \end{pmatrix} \sin(\omega t)$$

The solution could be obtained using method of undetermined coefficients.

The solution is another vector which could be shown as:

$$X = \text{Im}\{(-\omega^2 M + i\omega C + K)^{-1} \cdot F\}, i = \sqrt{-1}$$

The reason why we took the imaginary part comes from the method of undetermined coefficients principle. The idea is that we assume $F = \text{Im}\{F_0 \cdot e^{i\omega t}\}$. As a result, the solution should be in the form of the forcing term with a phase difference. However, in this report, we will focus on the results rather than these details.

It should be pointed out that we are only dealing with the amplitude of the response. To obtain the amplitude, we can take the absolute of the total response, which can be expressed as:

$$X_0 = \text{abs}\{(-\omega^2 M + i\omega C + K)^{-1} \cdot F\}$$

After modelling and solving the system analytically, it is easy to code this solution in MATLAB.

MATLAB scripts will be provided in appendices. Let's see the plots of the system. Firstly, we can take a look at the system without any tuned mass damper. To make things easier, we can take

$$m_1 = 1 \text{ kg}, k_1 = 1 \frac{\text{N}}{\text{m}}, c_1 = 0.1 \frac{\text{Ns}}{\text{m}}, m_2 = 0 \text{ kg}, F_1 = 1 \text{ N}, F_2 = 0 \text{ N}$$

After taking $m_2 = 0 \text{ kg}$, k_2 and c_2 can be picked arbitrarily since they will have no effect on the host structure. The plot is shown below in **Figure 2**.

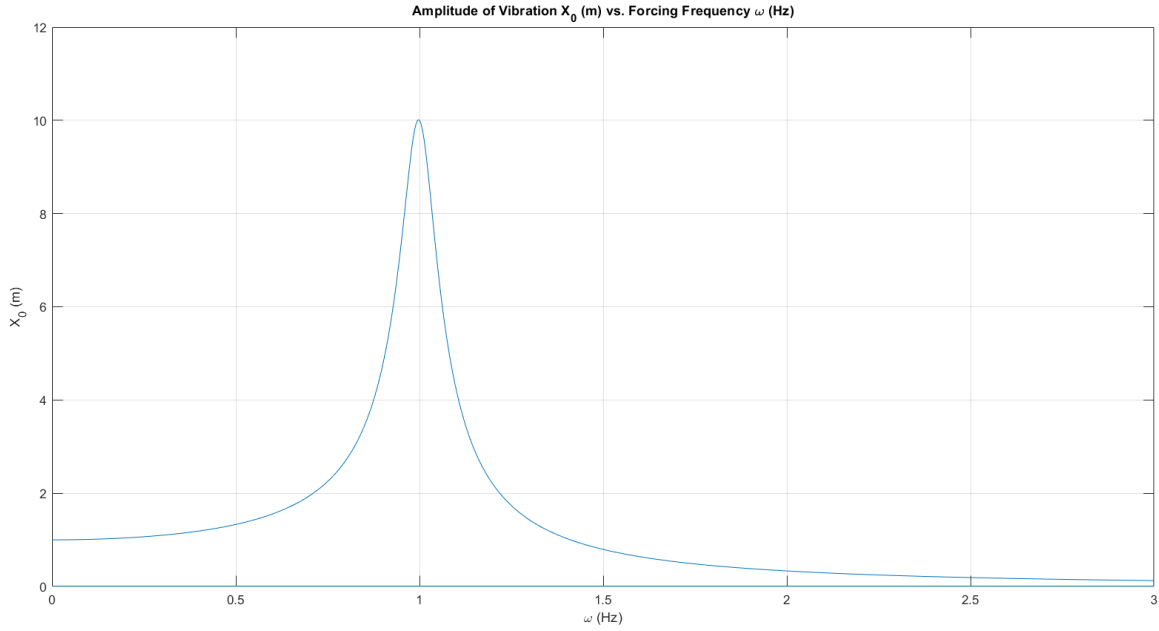


Figure 2: Single Degree of Freedom System with Harmonic Disturbance

As we can see from the plot, the system has a peak around 1 Hz. This is expected because the system has a natural frequency of $\omega_n = \sqrt{\frac{k_1}{m_1}} = 1 \text{ Hz}$. It should be noted that the peak does not occur at exactly $\omega = 1 \text{ Hz}$ but around that value.

2.1.1 Optimization for 2-DOF System

The main is to reduce the amplitude of the mechanical vibrations of the host structure. This can be done in many ways. A simple and uneducated solution would be to increase m_2 as much as possible so that system will not vibrate. However, this solution is far from an engineering sense. This is why, we will limit our absorber mass $m_2 = 0.1 \text{ kg}$ which is 10% of the main structure's mass. When the absorber's mass is limited,

we should find better ways to reduce vibration amplitude. The first one is **H_2 optimization**. This method focuses on kinetic energy reduction. To achieve this, we numerically integrate our plot using Simpson's 1/3 Rule and try to find the optimal values of k_2 and c_2 which would minimize the integral value.

H_2 Optimization

Let's see the plot first in **Figure 3**.

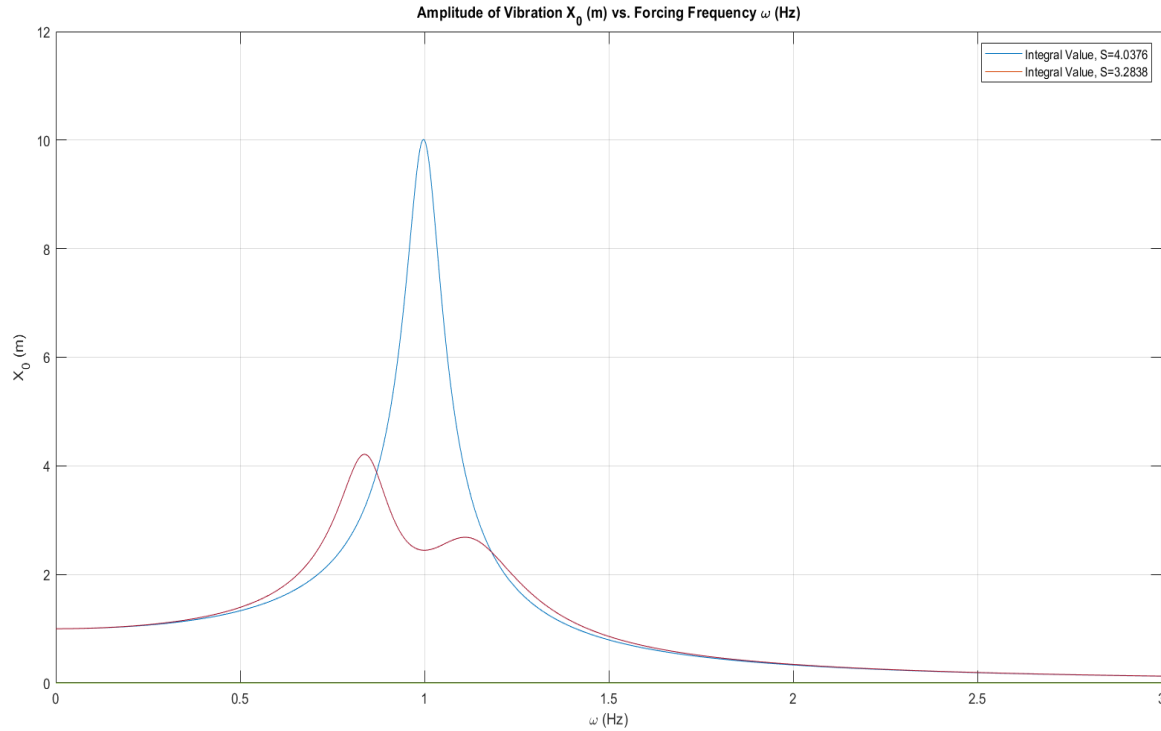


Figure 3: H_2 optimization for 2-DOF System

As we can see from the graph, our peak performance improved significantly. When the main structure is not treated, its peak value is around 10 meters but when an absorber with 0.1 kg is added to the structure, its maximum peak is around 4.1-4.3 meters. Also, we can see by numerical integration that integral value, S , under the curve between $\omega = 0 \text{ Hz}$ to $\omega = 3 \text{ Hz}$ decreases from 4.0376 to 3.2838. It is obvious that our system has improved a lot. To minimize this integral value, S , we used MATLAB built-in functions.

As explained before, we integrate our amplitude graph with respect to forcing frequency numerically using Simpson's 1/3 Rule. This integration is done by a function written by us. After that, we put out integration function, let's call *integrate*, into a MATLAB built-in function *fminsearch*. There are plenty of

built-in functions in MATLAB to do multi-dimensional optimization. For now, *fminsearch* is pretty efficient and proper for us.

To sum up, we optimize our result by *fminsearch* which gives us a vector. In this vector, the optimum k_2 and c_2 values are stored. For our case, optimum values are:

$$k_2 = 0.0915 \frac{N}{M}, c_2 = 0.03 \frac{Ns}{m}$$

H_∞ Optimization

The second method is H_∞ optimization. This method focuses on reducing and equating the peak values. Before the discussion, let's see the plot in **Figure 4**.

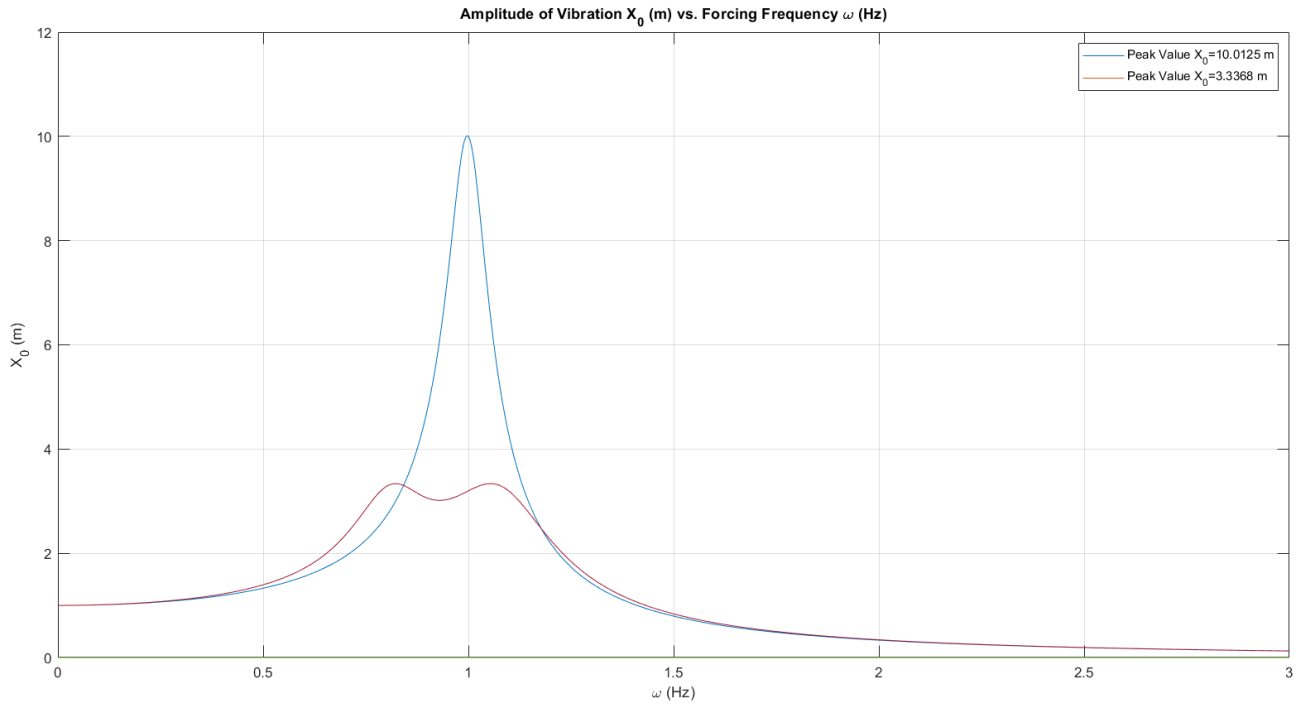


Figure 4: H_∞ optimization for 2-DOF System

The algorithm for this optimization is a little bit different from the algorithm for H_2 optimization.

Firstly, we designed an algorithm to get peak values of the amplitude. To do this, we used golden section search algorithm. Golden section search can find the peak values very fast. However, to find the peaks, golden section search needs brackets. What this means is that if we can bracket

the peak values, golden section search can find peak location very easily. An educated engineer can easily see that peaks must occur around the natural frequencies of the system. This gives us a bound. From this point, we can simply check if the function is increasing or decreasing. If the function is decreasing, we go to left to find the other bound. Otherwise, if the function is increasing, we go right to find the other bound. Now that the bounds are obtained, we leave the finding peaks part to the golden section search.

To use golden section search, we need natural frequencies but how do we find them? Natural frequencies are square roots of the eigenvalues of the free undamped system. To give more details, we can express undamped free system as:

$$M\ddot{x} + Kx = 0$$

M and K are same as before. Thus, natural frequencies can be expressed as:

$$\omega_{1,2} = \sqrt{\text{eig}(M^{-1} \cdot K)}$$

Secondly, we use MATLAB built-in function again *fminsearch*. From above, we obtained 2 peak locations (since our system is 2-DOF system) as a 2×2 matrix. In the first row we have ω values, in the second row we have X_0 values. We create another function, call *maxofpeaks*, which takes the maximum of the second row. We put *maxofpeaks* into the *fminsearch*. MATLAB function *fminsearch* tries to minimize peaks. One should be careful, we did not have to create any algorithm for equating because our *maxofpeaks* function is created elegantly. This function always takes the maximum and *fminsearch* always tries to minimize it. If both X_0 values are equal, then our function, *maxofpeaks*, will not return anything and *fminsearch* will stop.

From **Figure 4** we can see that maximum peak value is around 2.6-2.7 meters. This indicates that our peak performance improved even more. We can check the integral value for H_∞ optimization from our MATLAB code and see that $S = 3.3067$ which lower than the value for H_2 optimization ($S = 3.2838$). This result is also expected since we did not try to optimize our tuned mass damper system for minimum kinetic energy absorbtion of the host structure.

To wrap up, 2 optimization methods are implemented to the tuned mass damper system which are H_2 and H_∞ . H_2 method focuses on the reduction of the kinetic energy delivered to the host structure. To implement H_2 method, we created a function called *integrate*. What this function does is that it integrates the amplitude of vibration (X_0) with respect to forcing frequency (ω) numerically using Simpson's 1/3 Rule. After that we used MATLAB built-in function called *fminsearch* to find the minimum value of this integral. This function, *fminsearch*, returns a vector. This vector stores corresponding k_2 and c_2 values(in our case, $k_2 = 0.0915 \frac{N}{m}$, $c_2 = 0.03 \frac{Ns}{m}$). On the other hand, H_∞ method focuses on the reduction of the peak values of the host structure. To implement H_∞ method we created 3 different functions. The first one is *natfreq*. This function obtains the natural frequencies (ω_n) of the system by solving the **eigenvalue problem(EVP)** and taking the square roots

of eigenvalues. The second function is *getpeaks*. This function uses the outputs of the *natfreq* function and brackets the peak values of the system (we know that peaks must occur around natural frequencies). After bracketing, *getpeaks* uses a sophisticated method for finding peaks which is called golden section search. Function *getpeaks* returns a 2×2 matrix. In the first row, ω values are stored. In the second row, X_0 values are stored. The final function is called *maxofpeaks*. This function simply takes the second row of *getpeaks* function's output and returns maximum of these values. The rest is done by *fminsearch* which returns a vector storing k_2 and c_2 . In our case,

$$k_2 = 0.0788 \frac{N}{m} \text{ and } c_2 = 0.0342 \frac{Ns}{m}.$$

We've come this far and explained the process. In the next sections, we will use similar approaches. However, there is a certain change in the application which is *fminsearch*. We will use another MATLAB built-in function *fmincon*. The reason is that our system will be more complicated (3-DOF systems), so we need to define some upper and lower bounds for our solutions. *fmincon* allows us to define these and some additional bounds.

2.2 Three Degree of Freedom Systems with a Harmonic Disturbance

2.2.1 3-DOF Systems with Parallel Connection

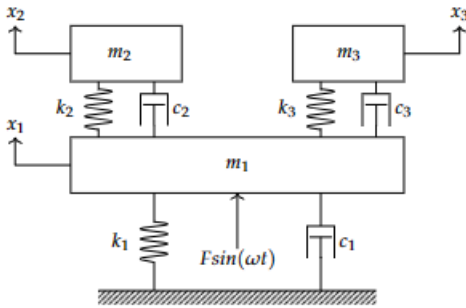


Figure 5: 3-DOF System with Parallel Connection

The system can be seen in **Figure 5**. We added another tuned mass damper system to our host structure. The discussion regarding the mass of the tuned mass dampers are made above. Therefore, we can simply put a constraint such that,

$$m_2 + m_3 = 0.1 \text{ kg}$$

This is 10% of the main structure. We will now update our code to find the values of k_2, c_2, m_2, k_3, c_3 , and m_3 . We have to define a physical constrain to be efficient.

$$k_2, c_2, m_2, k_3, c_3, m_3 \geq 0$$

Solution procedure is the same as before but some matrices and vectors are changed. Shown as,

$$M = \begin{bmatrix} m_1 & 0 & 0 \\ 0 & m_2 & 0 \\ 0 & 0 & m_3 \end{bmatrix}, C = \begin{bmatrix} c_1 + c_2 + c_3 & -c_2 & -c_3 \\ -c_2 & c_2 & 0 \\ -c_3 & 0 & c_3 \end{bmatrix}, K = \begin{bmatrix} k_1 + k_2 + k_3 & -k_2 & -k_3 \\ -k_2 & k_2 & 0 \\ -k_3 & 0 & k_3 \end{bmatrix}$$

Also,

$$\ddot{x} = \begin{pmatrix} \ddot{x}_1 \\ \ddot{x}_2 \\ \ddot{x}_3 \end{pmatrix}, \dot{x} = \begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{pmatrix}, x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}, F = \begin{pmatrix} F_1 \\ 0 \\ 0 \end{pmatrix} \sin(\omega t)$$

It is explicit that we need to optimize 6 parameters($k_2, c_2, m_2, k_3, c_3, m_3$). This needs some considerable computing power(compared to 2-DOF system). Thus, we have to be more precise with our initial guesses. MATLAB built-in functions for optimization need initial guesses to start with (*fminsearch, fmincon* etc.). This was the case for 2-DOF system but the algorithm is more robust when it is 2-D optimization. 6-D optimization is more sensitive, so we give initial guesses around 0.1 for each parameter. A justification for these guesses is needed without any doubts which would be the 10% rule for us. Obviously, this rule is not reliable at all. Furthermore, in more complex cases, this rule would crash and would not even work. With all due respect, our cases are generally simple. A simple rule, which is consistent with common sense, will not hurt.

2.2.1.1 Optimization

H_2 Optimization

To be consistent, we will follow the same structure above. Let's take a look at the plot of H_2 optimization in Figure 6.

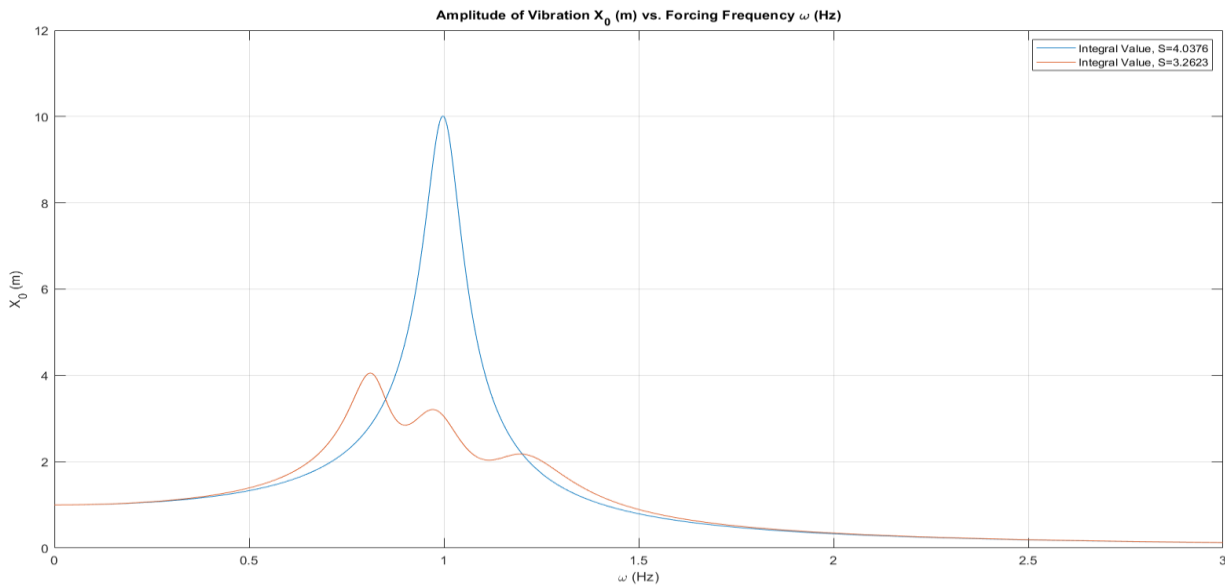


Figure 6: H_2 optimization for 3-DOF System with Parallel Connection

The kinetic energy delivered to the main structure ($S = 3.2623$) is less than the value that is obtained from 2-DOF system ($S = 3.2838$ for H_2 optimization). There is an improvement which costs more computing power and time. There is a tradeoff, as in all of the engineering applications. We can choose 2-DOF or 3-DOF system according to our need of precision. The gain is 0.65% which appears to be small. Nevertheless, we divided the tuned mass damper system parameters. If it is hard for us to obtain 10% of the main structure, we can use smaller pieces and still achieve an excellent performance (even better).

The results are,

$$\begin{aligned} k_2 &= 0.0591 \frac{N}{m}, & c_2 &= 0.0113 \frac{Ns}{m}, & m_2 &= 0.0489 \text{ kg}, \\ k_3 &= 0.0395 \frac{N}{m}, & c_3 &= 0.0084 \frac{Ns}{m}, & m_3 &= 0.0511 \text{ kg} \end{aligned}$$

H_∞ Optimization

As mentioned earlier, our optimization algorithm is very sensitive to initial guesses. We could obtain some reasonable results with 10% rule using H_2 optimization. The results of H_2 optimization are a perfect fit for the initial guesses of H_∞ optimization. Using these initial guesses, let's take a look at the plot of H_∞ optimization in **Figure 7**.

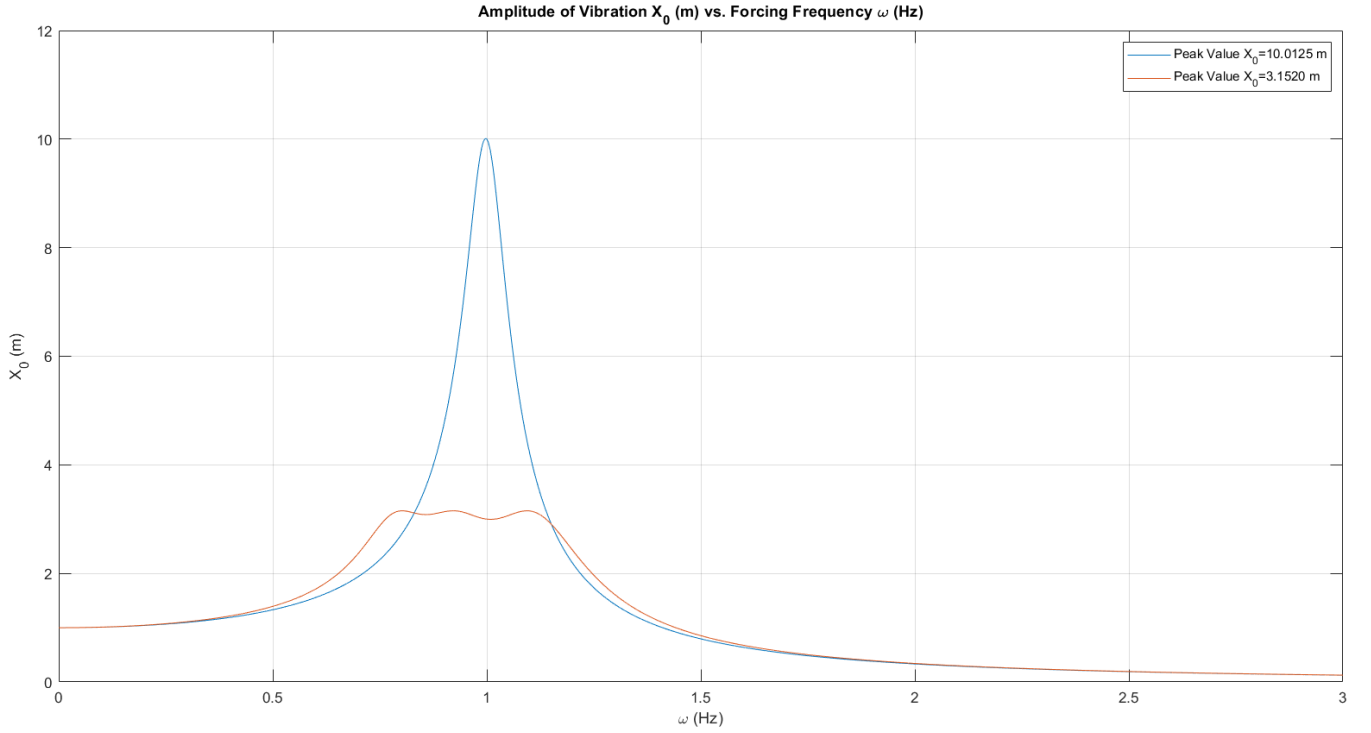


Figure 7: H_∞ optimization for 3-DOF System with Parallel Connection

The peak is $X_0 = 3.1520$ m which is less than the value of 2-DOF system $X_0 = 3.3368$ m. The gain is 5.5%. This time, our gain is considerably high. The discussion is similar. One can choose the system according to the system and precision needs.

2.2.2 3-DOF System with Series Connection

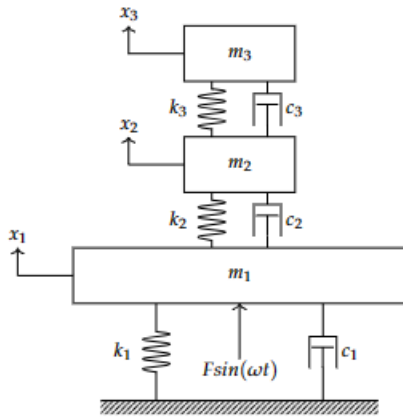


Figure 8: 3-DOF System with Series Connection

The system can be seen in **Figure 8**. There are minimal changes to the solution. Similar to the previous case, we have some different matrices. Shown as,

$$C = \begin{bmatrix} c_1 + c_2 & -c_2 & 0 \\ -c_2 & c_2 + c_3 & -c_3 \\ 0 & -c_3 & c_3 \end{bmatrix}$$

$$K = \begin{bmatrix} k_1 + k_2 & -k_2 & 0 \\ -k_2 & k_2 + k_3 & -k_3 \\ 0 & -k_3 & k_3 \end{bmatrix}$$

Only changes are presented above. Remaining matrix and vectors $(M, \ddot{x}, \dot{x}, x, F)$ are the same with 3-DOF Systems with Parallel Connection. We have the same considerations with 3-DOF Systems with Parallel Connection. Since they are mentioned earlier, we can jump into the optimization plots and comment on them.

2.2.2.1 Optimization

H_2 Optimization

Let's see the plot in **Figure 9**.

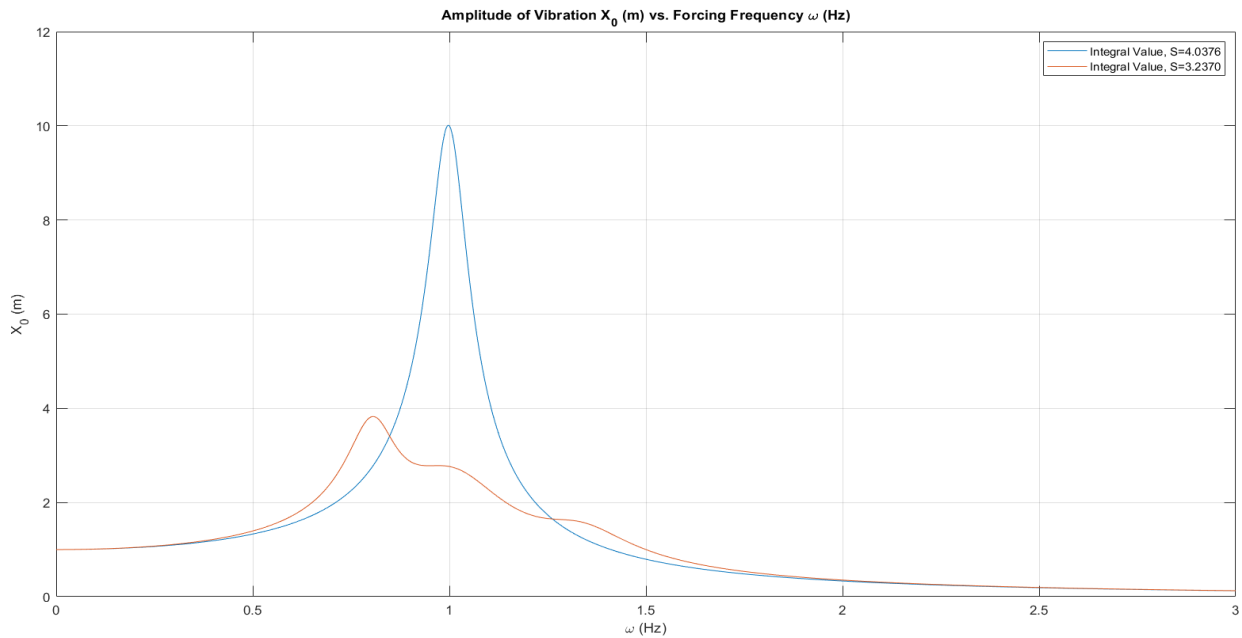


Figure 9: H_2 Optimization for 3-DOF System with Series Connection

Peak performance improved. Also, the integral value is decreased ($S = 3.2370$). This connection type outputs enhanced results with H_2 optimization. The drawback can be the longitudinal space requirement. Moreover, we analyze the system in direction (x direction) but this type of construction may be fragile to other types of vibration.

It should be noted that the results of H_2 optimization for 3-DOF systems with parallel connection are used as initial guesses for this algorithm. Results are,

$$k_2 = 0.1111 \frac{N}{m}, c_2 = 0 \frac{Ns}{m}, m_2 = 0.0809 \text{ kg}$$

$$k_3 = 0.0172 \frac{N}{m}, c_3 = 0.0093 \frac{Ns}{m}, m_3 = 0.0191 \text{ kg}$$

Optimum value of c_2 is $0 \frac{Ns}{m}$. This result is interesting. Not using any damper for the second mass is the most efficient way for this connection and this optimization method.

H_2 Optimization

The plot is in **Figure 10**.

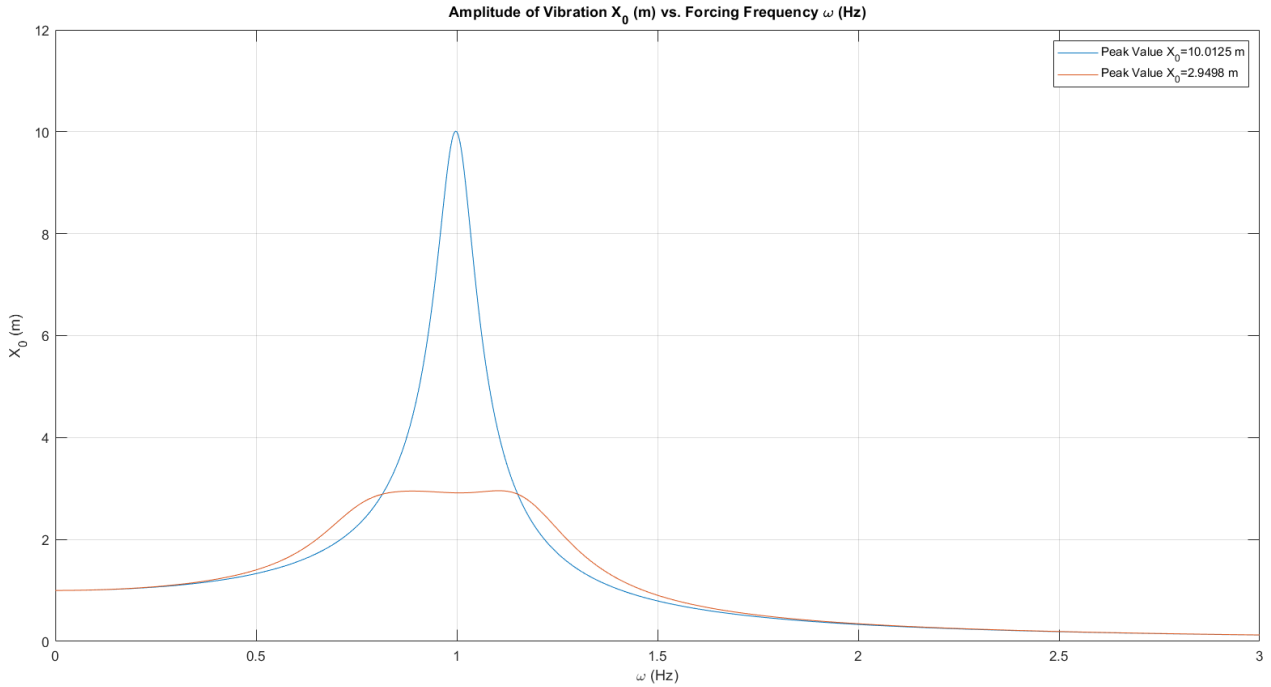


Figure 10: H_2 Optimization for 3-DOF System with Series Connection

Additional Discussion

There is a phenomenon that can be interesting. When we were doing our research, we found out that the vibration at the exact natural frequency (ω_n) of the host structure can be eliminated wholly. However, there is a trade-off between peaks of displacement and minimum displacement at the natural frequency; from **Figure 11**, the trade-off can be seen very clearly. This property seems pretty useful, but it can be very dangerous. The reason behind this is the sensitivity of the forcing frequency. It is impractical for most of the cases to keep the forcing frequency at a certain value, which must be the natural frequency of the host structure(ω_n). For example, a working machine or a vibrating structure passes a peak until the

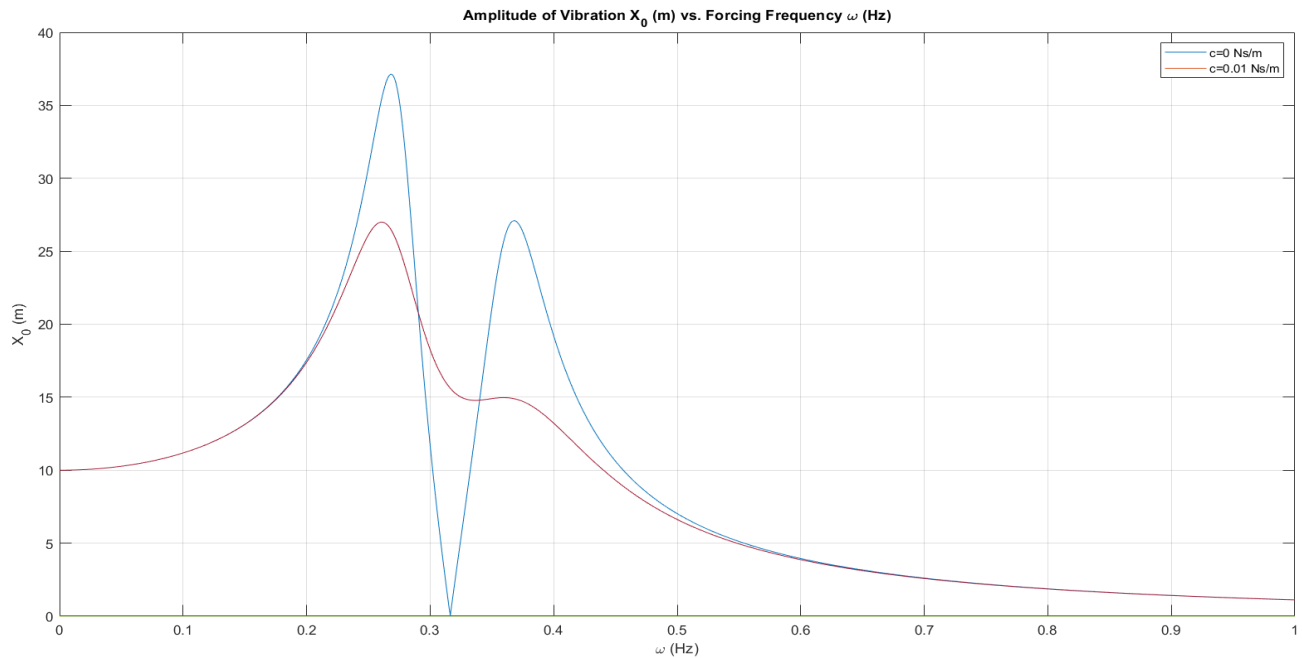


Figure 11: 2-DOF System with Different Damping (c) Values

forcing frequency reaches a certain value. Since the first peak is pretty high, the result can be catastrophic. If the forcing frequency cannot be kept at a certain value, the host structure will reach another peak suddenly. Thus, some damping makes the vibration isolation more controllable and practical.

3 Demo Project

For the project, we have been assigned to implement the tuned mass dampener approach to reduce the vibration amplitudes of a horizontally vibrating system. The sections below explain the construction, analysis, operation, and results of the 2-DOF vibrating system.

3.1 Physical Model

The physical model can be seen in Figure 12. The basic 2-DOF vibratory system is composed of 3 steel rulers, 12 neodymium magnets, a copper plate, and a cardboard base. 2 rulers are attached to the cardboard base, and the free ends are put through a wooden base. The rulers are attached to the wooden base with the use of a structure comprised of wooden tongue depressors, which allow vertical repositioning of the rulers, which in turn allows the arrangement of the height of the base plate. The two parallel rulers only allow horizontal movement of the cardboard base due to their positioning. The cardboard base and the two rulers are the primary structure. The third ruler is attached to the cardboard base with a similar approach as the two rulers, which allows vertical repositioning and horizontal movement of the ruler. At the free tip of the 3rd ruler, magnets, and coins are attached as additional mass. A copper plate is extended to the end of the 3rd ruler from the cardboard base with a rigid wooden structure. When paired with moving magnets at the tip of the ruler, this adds a linear damping component to the secondary structure. The masses, the ruler, and the copper-magnet interaction create the secondary structure (TMD).

The the cardboard base is equipped with several electronic devices. At the center of the base a DC motor wheel is attached and on top of the wheel, 2 coins are attached away from the rotation axis of the wheel. This creates an unbalance when the wheel is rotating which creates horizontal harmonic forcing at the cardboard base with frequency equal to the rotational speed of the wheel. The rotational speed of the wheel is measured with an IR color sensor attached close to the wheel. An accelometer is also attached to the base structure to determine vibrational properties of the base structure. Mass of all these components (as well as the copper plate and its structure) are a part of the primary structure.

When simplified, this structure can be modeled as a 2-DOF vibrating system with the main stiffness components being the rulers.



Figure 12: The Physical System

3.2 Analysis of Base Structure

In order to perform the optimization algorithm on this system, the vibrational parameters of the primary structure had to be found. In order to achieve this, the logarithmic decrement method was used on the base structure without the secondary structure attached.

Logarithmic decrement is a technique used in vibration analysis to determine the vibrational properties of a structure. When free vibration response data of an underdamped structure is available, one can use the logarithmic decrement method. The method uses amplitudes of successive peaks to determine the logarithmic decrement:

$$\delta = \frac{1}{n} \ln \frac{x(t)}{x(t+nT)} = \frac{1}{n} \ln \frac{\dot{x}(t)}{\dot{x}(t+nT)} = \frac{1}{n} \ln \frac{\ddot{x}(t)}{\ddot{x}(t+nT)}$$

n : number of cycles

$x(t), \dot{x}(t), \ddot{x}(t)$: amplitude of peak at time t

$x(t+nT), \dot{x}(t+nT), \ddot{x}(t+nT)$: amplitude of peak n cycles later from first peak

The damping ratio then can be calculated from:

$$\xi = \frac{\delta}{\sqrt{4\pi^2 + \delta^2}}$$

Damped & undamped natural frequency can be calculated from the free vibration response and the damping ratio:

$$\omega_d = \frac{2\pi}{T}$$

$$\omega_n = \frac{\omega_d}{\sqrt{1 - \xi^2}}$$

Damping and stiffness values can be found if the mass of the system is known. In order to find the effective mass of the system, the mass of the cardboard base is measured with the components attached to it (Without the rulers). Then, the effective mass corresponding to the active length of the rulers (17 cm) is added to the cardboard mass, which is found using the relation:

$$m_{eff} = 0.24 \cdot m_{act}$$

With the main structure's mass found, one can calculate stiffness and damping coefficients using the relations:

$$k_1 = \omega_n^2 \cdot m_1$$

$$c_1 = 2 \cdot m_1 \cdot \omega_n$$

For our system, we used data acquired from the accelerometer to perform the logarithmic decrement method. The primary structure is given an initial deflection and is then left free to vibrate. While the structure is vibrating, accelerometer data is collected with the use of an Arduino Uno board. Data collected from the accelerometer is noisy; in order to filter the signal, data is sent to Python, where a 2nd order Butterworth filter is applied, which removes high-frequency noise from raw accelerometer data. Raw and filtered data for free vibration of the primary structure is given in **Figure 13**. Arduino and Python code for operation, data collection, transfer, filtering, and logarithmic decrement are supplied in the **appendix**.

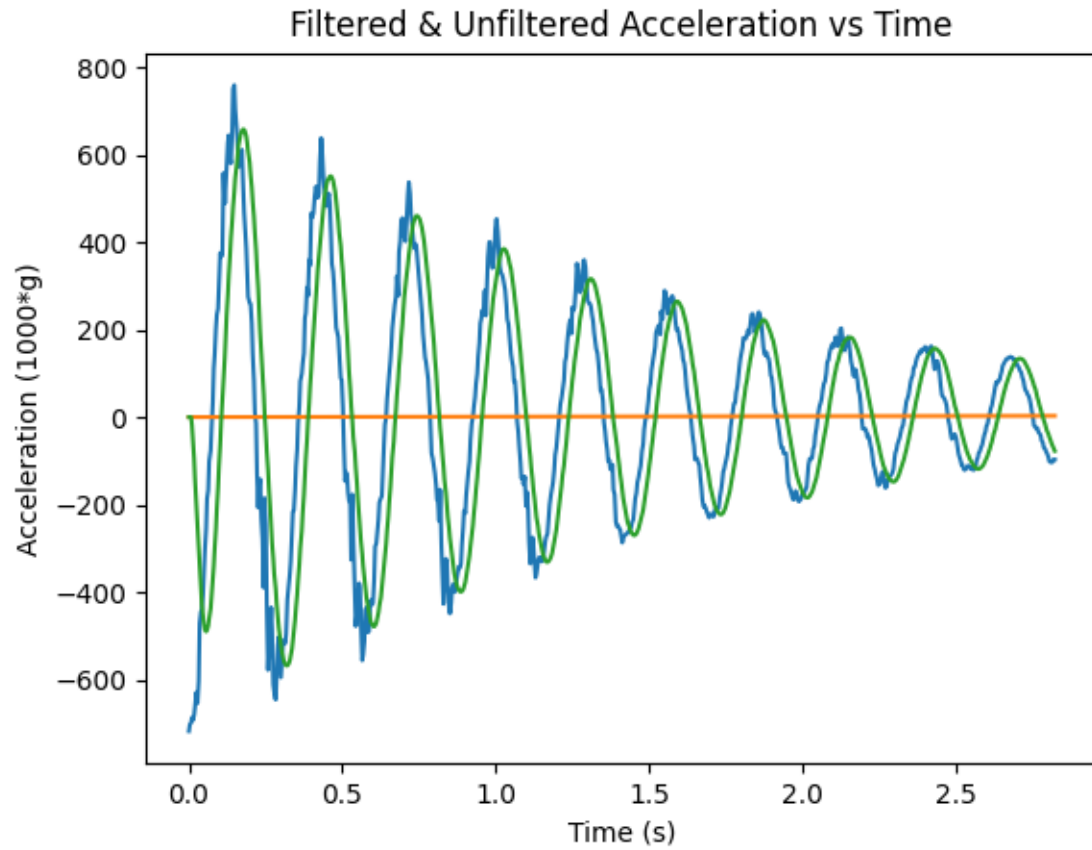


Figure 13: Filtered & Unfiltered Acceleration vs Time

Logarithmic decrement is performed on the filtered data, and vibrational parameters of base structure are calculated from logarithmic decrement value and known effective mass. The results are given below:

$$m_1 = 0.249 \text{ kg}$$

$$\xi = 0.0329$$

$$\omega_d = 22.307 \text{ rad/sec}$$

$$\omega_n = 22.295 \text{ rad/sec}$$

$$k_1 = 123.538 \text{ N/m}$$

$$c_1 = 0.365 \text{ Ns/m}$$

Note that the damping ratio is very low since the only active component of damping is structural damping for the primary structure.

3.3 Construction and Optimization of TMD

Theory suggests that when the unbalanced rotational speed reaches the damped natural frequency of the base structure (without TMD), the base structure will enter resonance. When the base structure is put through a frequency sweep with the unbalanced motor, the maximum amplitude of oscillations is seen around the damped natural frequency value of the base structure. This result showed us that our method and calculations for the properties of the base structure were valid. Now, we could start the optimization process for the TMD.

The optimization of choice for the 2-DOF system was the optimization. As mentioned in previous sections, initial guesses and constraints for the optimization are key for a well-optimized system. The lumped mass for the TMD was constrained to be less than 14% of the base structure mass. To establish proper constraints and a proper initial guess for the stiffness value of the TMD (Ruler), the stiffness value range for different ruler lengths had to be found. In order to do this, we have performed several static deflection tests as well as a logarithmic decrement method to relate the ruler length to the stiffness value of the ruler. It is known that the stiffness values for a cantilever beam are inversely proportional to the cube of the active length of the ruler. So, after several experiments, we were able to achieve stiffness values between 100 N/m and 10 N/m within reasonable ruler lengths for proper TMD operation.

Initial tests for the 2-DOF structure were performed without damping of the TMD. As an initial test, we tuned the mass and stiffness of the TMD such that the natural frequency of the TMD and the base structure were the same. As mentioned in **Figure 11**, in the undamped TMD case, we should expect almost no movement at the base structure when the unbalance is rotating at the damped natural frequency of the base structure. Our expectations were met. The undamped case also resulted in 2 significant vibration amplitudes before and after the resonant frequency. Initially, we tried optimizing the structure with almost no TMD damping, but this proved to be insignificant, as expected. Damping of the TMD was a must for proper vibration isolation.

Damping had to be introduced to the TMD while still satisfying the linearity requirement of the equation of motion. With the guidance of our professor and through a literature survey, we have opted for a technique called “Eddy Current Breaking.” The basic principle is when a magnetic field passes through a conductor; eddy currents are created inside the conductor that oppose the change of the magnetic flux through the conductor. These local eddy currents create forces opposing the motion of the magnetic fields. The strength of the damping forces created by the eddy currents is directly proportional to the speed of the magnetic field passing through the conductor. As a result, eddy current breaking creates a forcing term that opposes the motion of a magnet proportional to the speed of the magnet (viscous damping).

When the vibrational properties of the TMD were calculated through the logarithmic decrement method, we observed a relatively high damping ratio, indicating that our damper was working properly. Due to the construction of our system, it was hard to control the distance between the plate and the magnets exactly. This removed our ability to precisely control the damping coefficient. So, we constrained the damping value in the algorithm to be around the average value we received from various logarithmic decrement tests for a fixed distance between the magnet and the plate. This meant that the optimization would have to be performed on the mass and stiffness. The results of the optimization algorithm are provided below:

$$m_2 = 0.033 \text{ kg}$$

$$k_2 = 12.23 \text{ N/m}$$

$$c_2 = 0.295 \text{ Ns/m}$$

The plot of the optimization results are provided in **Figure 14**.

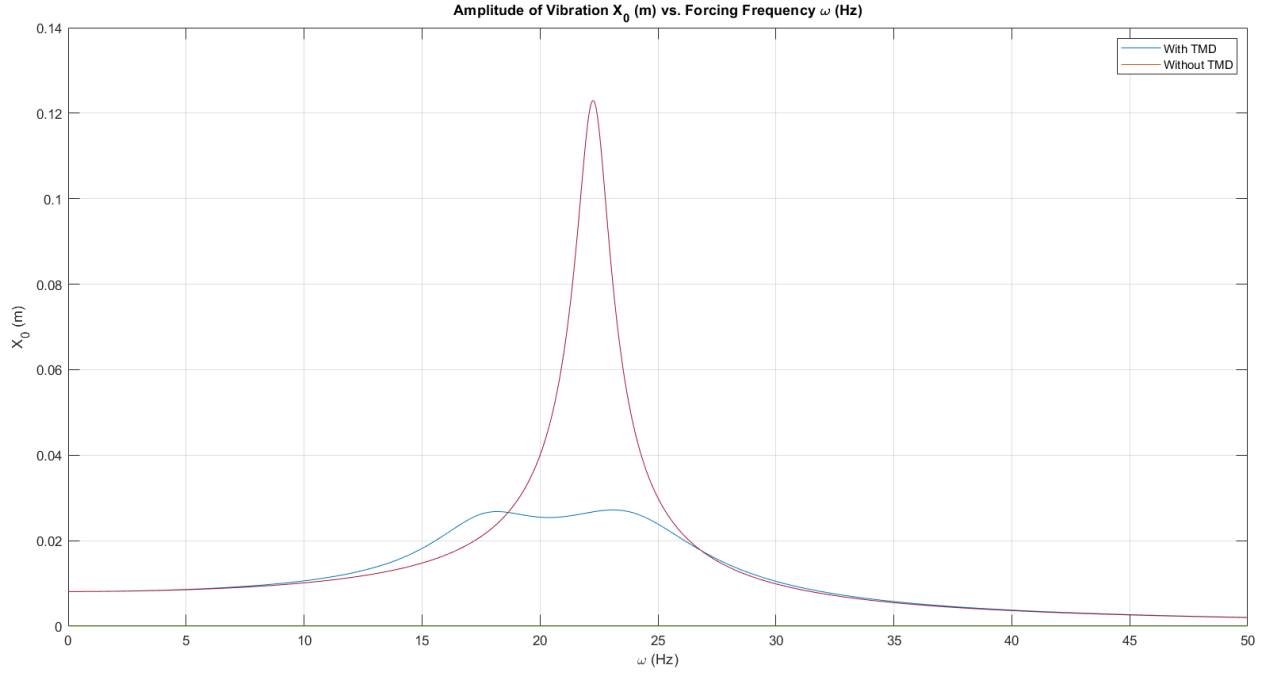


Figure 14: Amplitude Response of the Structure to Varying Forcing Frequencies

3.4 Results and Conclusion

Once the optimization was completed, we set up the TMD such that the mass and the stiffness values correspond to that of the optimization output. The results were highly satisfactory. Even though we didn't have a proper method of measuring vibration amplitude, the difference in vibration amplitudes of the structure with and without the TMD was obviously visible to the naked eye. We were also able to observe the two peaks resulting from the addition of the TMD. Our result was similar to **Figure 14** but not exact. The reason for this is that we weren't able to precisely adjust the damping of the TMD.

4 Appendix

4.1 MATLAB Code for Optimization

```
clear
clc

m1 = 1; %kg
k1 = 0.1; %N/m
c1 = 0.1; %Ns/m
m2 = 0.1; %kg
F_1 = 1; %N
F_2 = 0;
F_3 = 0;
%% 3 DOF Series Integral Approach
% t=3;
% w = linspace(0,3,1000);
% x0=[0.0591,0.0113,0.0489,0.0395,0.0084,0.0511]; % [k2,c2,m2,k3,c3,m3]
% y=[m1 k1 c1 F_1 F_2 F_3];
% A=[0,0,1,0,0,1];
% b=0.1;
% lb=[0,0,0,0,0,0];
% ub=[1,1,1,1,1,1];
% Aeq=[];
% beq=[];
% opt_3dof=@(x)integrate(x,y,w,t);
% opt_points = fmincon(opt_3dof,x0,A,b,Aeq,beq,lb,ub);
% plot(w,tmd(opt_points,y,w,t))
% s=integrate(opt_points,y,w,t);
% title("Amplitude of Vibration X_{0} (m) vs. Forcing Frequency \omega (Hz)")
% ylabel("X_{0} (m)")
% xlabel("\omega (Hz)")
% hold on
% legend("Integral Value, S=4.0376","Integral Value, S=3.2370")
% grid on

%% 3 DOF Series Equating Peaks
% t=3;
% w = linspace(0,3,1000);
% y = [m1 k1 c1 F_1 F_2 F_3];
% x0=[0.11,0,0.08,0.017,0.0093,0.05]; % [k2,c2,m2,k3,c3,m3]
% A=[0,0,1,0,0,1];
% b=0.1;
% lb=0*[0.001,0.001,0.001,0.001,0.001,0.019];
% ub=[];
% Aeq=[];
% beq=[];
% opt_3dofeq=@(x)maxofpeaks(x,y,w,t);
% r = fmincon(opt_3dofeq,x0,A,b,Aeq,beq,lb,ub);
% plot(w,tmd(r,y,w,t))
% % get_peaks(r,y,w,t)
% title("Amplitude of Vibration X_{0} (m) vs. Forcing Frequency \omega (Hz)")
% ylabel("X_{0} (m)")
% xlabel("\omega (Hz)")
% hold on
% legend("Peak Value X_0=10.0125 m","Peak Value X_0=2.9498 m")
% grid on

%% 3 DOF Parallel Equating Peaks
% t=2;
% w = linspace(0,3,1000);
% y = [m1 k1 c1 F_1 F_2 F_3];
% x0=[0.0591,0.0113,0.0489,0.0395,0.0084,0.0511]; % [k2,c2,m2,k3,c3,m3]
%
% A=[0,0,1,0,0,1];
```

```

% b=0.1;
% lb=[0.001,0.001,0.001,0.001,0.001,0.001];
% ub=[];
% Aeq=[];
% beq=[];
%
% opt_3dof=@(x)maxofpeaks(x,y,w,t);
% r = fmincon(opt_3dof,x0,A,b,Aeq,beq,lb,ub);
% i=integrate(r,y,w,t);
% p=get_peaks(r,y,w,t);
% plot(w,tmd(r,y,w,t))
%
% title("Amplitude of Vibration X_{0} (m) vs. Forcing Frequency \omega (Hz)")
% ylabel("X_{0} (m)")
% xlabel("\omega (Hz)")
% hold on
% legend("Peak Value X_0=10.0125 m","Peak Value X_0=3.1520 m")
% grid on

%% 3 DOF Parallel Integral Approach
% t=2;
% w = linspace(0,3,1000);
% x0=[0.1,0.1,0.1/2,0.1,0.1,0.1/2]; % [k2,c2,m2,k3,c3,m3]
% y=[m1 k1 c1 F_1 F_2 F_3];
%
% A=[0,0,1,0,0,1];
% b=0.1;
% lb=[0.001,0.001,0.001,0.001,0.001,0.001];
% ub=[];
% Aeq=[];
% beq=[];
% opt_3dof=@(x)integrate(x,y,w,t);
% opt_points = fmincon(opt_3dof,x0,A,b,Aeq,beq,lb,ub);
% r=integrate(opt_points,y,w,t);
% plot(w,tmd(opt_points,y,w,t))
%
% title("Amplitude of Vibration X_{0} (m) vs. Forcing Frequency \omega (Hz)")
% ylabel("X_{0} (m)")
% xlabel("\omega (Hz)")
% hold on
% legend("Integral Value, S=4.0376","Integral Value, S=3.2623")
% grid on
%

%% 2 DOF Equating Peaks
t=1;
w = linspace(0,1,1000);
y = [m1 k1 c1 m2 F_1 F_2];
x0=[0.01 0.0];
x1=[0.01 0.01];
% equate=@(x)maxofpeaks(x,y,w,t);
% r=fminsearch(equate,x0);
% peaks=get_peaks(r,y,w,t);
% integrate(r,y,w,t)
plot(w,tmd(x0,y,w,t))
title("Amplitude of Vibration X_{0} (m) vs. Forcing Frequency \omega (Hz)")
ylabel("X_{0} (m)")
xlabel("\omega (Hz)")
grid on
% legend("Peak Value X_0=10.0125 m","Peak Value X_0=3.3368 m")
hold on
plot(w,tmd(x1,y,w,t))
legend('c=0 Ns/m','c=0.01 Ns/m');

%% 2 DOF Integral
% t=1;
% x0=[0,0];
% y = [m1 k1 c1 m2 F_1 F_2];

```

```

% w = linspace(0,3,1000);
% opt = @(x)integrate(x,y,w,t);
% opt_points=fmincon(opt,x0);
% i=string(integrate(opt_points,y,w,t));
%
% plot(w,tmd(x0,y,w,t))
% title("Amplitude of Vibration X_{0} (m) vs. Forcing Frequency \omega (Hz)")
% ylabel("X_{0} (m)")
% xlabel("\omega (Hz)")
% grid on
% legend("Integral Value, S=4.0376","Integral Value, S=3.2838")
% hold on

function L=maxofpeaks(x,y,w,t)
peaks=get_peaks(x,y,w,t);
L=max(peaks(2,:));
end

function peaks=get_peaks(x,y,w,t)
w_n=natfreq(x,y,w,t);
peaks=zeros(2,length(w_n));
peakfinder=@(w)(-tmd(x,y,w,t));
for i=1:length(w_n)
    peaks(1,i)=fminsearch(peakfinder,w_n(i));
    peaks(2,i)=tmd(x,y,peaks(1,i),t);
end

end

function nat_freq=natfreq(x,y,~,t)
if t==1
    k2=x(1);
    c2=x(2);

    m1=y(1);
    k1=y(2);
    c1=y(3);
    m2=y(4);
    F_1=y(5);
    F_2=y(6);

    M = [m1 0;
         0 m2];

    K = [k1+k2 -k2;
        -k2 k2];
    nat_freq=sqrt(eig(M^(-1)*K));
elseif t==2
    k2=x(1);
    c2=x(2);
    m2=x(3);
    k3=x(4);
    c3=x(5);
    m3=x(6);

    m1 = y(1);
    k1 = y(2);
    c1 = y(3);
    F_1 = y(4);
    F_2 = y(5);
    F_3 = y(6);

    M = [m1 0 0;
         0 m2 0;
         0 0 m3];
    K = [k1+k2+k3 -k2 -k3;

```

```

        -k2 k2 0;
        -k3 0 k3];
    nat_freq=sqrt(eig(M^(-1)*K));
elseif t==3
    k2=x(1);
    c2=x(2);
    m2=x(3);
    k3=x(4);
    c3=x(5);
    m3=x(6);

    m1 = y(1);
    k1 = y(2);
    c1 = y(3);
    F_1 = y(4);
    F_2 = y(5);
    F_3 = y(6);

    M = [m1 0 0;
          0 m2 0;
          0 0 m3];

    K = [k1+k2 -k2 0;
          -k2 k2+k3 -k3;
          0 -k3 k3];
    nat_freq=sqrt(eig(M^(-1)*K));
end
end
%
% function dP_2=secderivative(x,y,w,t)
% h = 0.01;
% dP_2 = (tmd(x,y,w+h,t)-2*tmd(x,y,w,t)+tmd(x,y,w-h,t))/(h^2);
% end

function dP=derivative(x,y,w,t)
h = 0.001;
dP = (-tmd(x,y,w+2*h,t)+8*tmd(x,y,w+h,t)-8*tmd(x,y,w-h,t)+tmd(x,y,w-2*h,t))/(12*h);
end

function S=integrate(x,y,w,t)
x1=tmd(x,y,w,t);
a=w(1); % lower limit
b=w(end); % upper limit
n = 10000; % number of segments-even number
h = (b-a)/n; % step size
fa = x1(1);
fb = x1(end);
S = (h/3)*(fa+fb);
for i=1:1:n
    if mod(i,2)==0
        S = S + (h/3)*2*tmd(x,y,(w(1)+i*h),t);
    else
        S = S + (h/3)*4*tmd(x,y,(w(1)+i*h),t);
    end
end
end
end

function x1=tmd(x,y,w,t)
if t==1

    k2=x(1);
    c2=x(2);

    m1=y(1);
    k1=y(2);
    c1=y(3);

```

```

m2=y(4);
F_1=y(5);
F_2=y(6);

M = [m1 0;
      0 m2];
C = [c1+c2 -c2;
      -c2 c2];
K = [k1+k2 -k2;
      -k2 k2];

F = [F_1;
      F_2];
x1 = zeros(length(w));
for i=1:length(w)
    X = abs((-w(i)^2*M+1i*w(i)*C+K)^(-1)*F);
    x1(i)= X(1);
end

elseif t==2

    k2=x(1);
    c2=x(2);
    m2=x(3);
    k3=x(4);
    c3=x(5);
    m3=x(6);

    m1 = y(1);
    k1 = y(2);
    c1 = y(3);
    F_1 = y(4);
    F_2 = y(5);
    F_3 = y(6);

    M = [m1 0 0;
          0 m2 0;
          0 0 m3];
    C = [c1+c2+c3 -c2 -c3;
          -c2 c2 0;
          -c3 0 c3];
    K = [k1+k2+k3 -k2 -k3;
          -k2 k2 0;
          -k3 0 k3];

    F = [F_1;
          F_2;
          F_3];

    for i=1:length(w)
        X = abs((-w(i)^2*M+1i*w(i)*C+K)\F);
        x1(i)= X(1);
    end
end
elseif t==3

    k2=x(1);
    c2=x(2);
    m2=x(3);
    k3=x(4);
    c3=x(5);
    m3=x(6);

    m1 = y(1);
    k1 = y(2);
    c1 = y(3);
    F_1 = y(4);

```

```

F_2 = y(5);
F_3 = y(6);

M = [m1 0 0;
      0 m2 0;
      0 0 m3];
C = [c1+c2 -c2 0;
      -c2 c2+c3 -c3;
      0 -c3 c3];
K = [k1+k2 -k2 0;
      -k2 k2+k3 -k3;
      0 -k3 k3];

F = [F_1;
      F_2;
      F_3];

for i=1:1:length(w)
    X = abs((-w(i)^2*M+1i*w(i)*C+K)\F);
    x1(i)= X(1);
end
end
end

```

4.2 Arduino Code for Operation

```

#include <QTRSensors.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "SparkFun_MMA8452Q.h"

LiquidCrystal_I2C lcd(0x27, 16, 2); // set the LCD address to 0x27 for a 16 chars and 2 line display

const int analogInPin = A0; // Analog input pin that the potentiometer is attached to
int sensorValue = 0; // value read from the pot
int outputValue = 0; // value output to the PWM (analog out)

int seconds = 0;
unsigned long previousMillis = 0;

QTRSensors qtr;
const int sensorTol = 400;
const uint8_t SensorCount = 1;
uint16_t sensorValues[SensorCount];

MMA8452Q accel;

const float Pi = 3.14159;

void setaccprec(int val) {
    if (val == 1) {
        accel.standby();
        accel.writeRegister(CTRL_REG1, 0b00000100);
        accel.active();
    } else if (val == 0) {
        accel.standby();
        accel.writeRegister(CTRL_REG1, 0b00000000);
        accel.active();
    }
}

float readvaluesrealtime() {
    const int buffer = 800;
    int counter = 0;
    unsigned long prevmicros = 0;

    int xn;

    Serial.println("start"); //Python waits for this

    while (true) {
        if (counter >= buffer) {
            counter = 0;
            break;
        }
        //timingfunc(1);
        if ((micros() - prevmicros) >= 2500) { //tobechanged for samplingrate
            if (accel.available()) {

```

```

        xn = int(1000.0 * accel.getCalculatedX());
        prevmicros = micros();
        counter++;
        Serial.print(millis());
        Serial.print(",");
        Serial.println(xn);
    }
    //timingfunc(0);
}
}
String RxdByte;
while (true) {
    if (Serial.available()) {
        RxdByte = Serial.readStringUntil('\n');
        float num = RxdByte.toFloat();
        return num;
    }
}
}

bool iswhite(int sensorValues) {
    if (sensorValues < sensorTol) {
        return true;
    }
    return false;
}

double calcradps() {
    int rev;
    unsigned long millisPrev;
    unsigned long firstMilli;
    unsigned long zerospeedTimer;

    millisPrev = millis();
    while (true) //Starts the timer and rev counter when the sensor sees white
    {
        zerospeedTimer = millis();
        if (zerospeedTimer - millisPrev > 2000) {
            return double(0);
        }
        qtr.read(sensorValues);
        //Serial.println(sensorValues[0]);
        if ((iswhite(sensorValues[0]) && (sensorValues[0]>50))) {
            rev = 0;
            millisPrev = millis();
            firstMilli = millisPrev;
            break;
        }
    }

    while (rev < 3) {
        unsigned long millisCount = millis();

        while (millisCount - millisPrev > 100) {
            millisCount = millis();
            if (millisCount - millisPrev > 1000) {
                return double(0);
            }
            qtr.read(sensorValues);
            //Serial.println(sensorValues[0]);
            if (iswhite(sensorValues[0])) {
                millisPrev = millisCount;
                rev++;
            }
        }
    }
    long milliseconds = (millisPrev - firstMilli);
    double rps = double(rev) / (double(milliseconds) / double(1000));
    return rps * 2 * Pi;
}

void setup() {
    // configure the sensors
    qtr.setTypeRC();
    qtr.setSensorPins((const uint8_t[]){ 5 }, SensorCount);
    //qtr.setEmitterPin(2);

    Serial.begin(115200);

    // // Configure Timer 1 for PWM @ 25 kHz.
    // TCCR1A = 0; // undo the configuration done by...
    // TCCR1B = 0; // ...the Arduino core library
    // TCNT1 = 0; // reset timer
    // TCCR1A = _BV(COM1A1) // non-inverted PWM on ch. A
    // | _BV(COM1B1) // same on ch; B
    // | _BV(WGM11); // mode 10: ph. correct PWM, TOP = ICR1
    // TCCR1B = _BV(WGM13) // ditto

```

```

//      | _BV(CS10); // prescaler = 1
// ICR1 = 320;      // TOP = 320

// Set the PWM pins as output.
pinMode(9, OUTPUT);
pinMode(10, OUTPUT);

lcd.init();
lcd.setBacklight(1);

Wire.begin();
if (accel.init(SCALE_4G, ODR_800) == 0) {
  Serial.println("Not Connected. Please check connections and read the hookup guide.");
  while (1)
    ;
}
setaccprec(1);
}

void loop() {
  // read the analog in value:
  sensorValue = analogRead(analogInPin);
  //Serial.println(sensorValue);
  // map it to the range of the analog out:
  //Serial.println(sensorValue);
  outputValue = map(sensorValue, 0, 1023, 0, 127);
  analogWrite(9, outputValue);
  Serial.println(calcradps());
}

```

4.3 Arduino Code for Data Collection

```

#include <Wire.h> // Must include Wire library for I2C
#include "SparkFun_MMA8452Q.h" // Click here to get the library: http://librarymanager/All#SparkFun_MMA8452Q
#include <MsTimer2.h>
MMA8452Q accel; // create instance of the MMA8452 class

unsigned long starttf = 0; //For timing function
unsigned long timingfunc(int val) {

  if (val == 1) {
    starttf = micros();
  } else if (val == 0 && starttf != 0) {
    Serial.println(micros() - starttf);
    starttf = 0;
  }
}

void setaccprec(int val) {
  if (val == 1) {
    accel.standby();
    accel.writeRegister(CTRL_REG1, 0b00000100);
    accel.active();
  } else if (val == 0) {
    accel.standby();
    accel.writeRegister(CTRL_REG1, 0b00000000);
    accel.active();
  }
}

float readvaluesrealtime() {
  const int buffer = 800;
  const int samplinglength = 2;
  const int samplingrate = 400;
  int counter = 0;
  unsigned long prevmicros = 0;
  unsigned long start;

  int xn;

  Serial.println("start"); //Python waits for this

  while (true) {
    if (counter >= buffer) {
      counter = 0;
      break;
    }
    //timingfunc(1);
    if ((micros() - prevmicros) >= 2500) {
      if (accel.available()) {
        xn = int(1000.0 * accel.getCalculatedX());
        prevmicros = micros();
        counter++;
        Serial.print(millis());
        Serial.print(",");
        Serial.println(xn);
      }
      //timingfunc(0);
    }
  }
}

```

```

String RxdByte;
while (true) {
  if (Serial.available()) {
    RxdByte = Serial.readStringUntil('\n');
    float num = RxdByte.toFloat();
    return num;
  }
}
}
void setup() {
  Serial.begin(115200);
  pinMode(LED_BUILTIN, OUTPUT);
  Wire.begin();
  Wire.setClock(400000);
  if (accel.init(SCALE_4G, ODR_800) == 0) {
    Serial.println("Not Connected. Please check connections and read the hookup guide.");
    while (1)
      ;
  }
  setaccprec(1);
}
void loop() {
  //Serial.println("Start!");
  readValuesRealtime();
  while (true) {
    digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
    delay(250); // wait for a second
    digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage LOW
    delay(250);
  }
}

```

4.4 Python Code for Logarithmic Decrement

```

def logdec(heights, peakindices, x, cycles, rulerlenght):
    #find number of peaks
    n = len(peakindices)
    #get second peak magnitude
    A1 = heights[1]
    A2 = heights[1+cycles]
    logdec = 1/cycles*np.log(A1/A2)
    damprat = logdec/np.sqrt(4*np.pi**2+logdec**2)
    period = (x[peakindices[1+cycles]]-x[peakindices[1]])/cycles #in seconds
    dampednaturalfreq = 2*np.pi/period #in rad/s
    naturalfreq = dampednaturalfreq*np.sqrt(1-damprat**2) #in rad/s
    mass = 0.0304 + 30/32.5*rulerlenght*10**-3*0.24 #in kg
    stiffness = mass*naturalfreq**2 #in N/m
    damping = 2*damprat*naturalfreq*mass #in Ns/m
    #return damping ratio, natural frequency, damped natural frequency, period, stiffness,
    #damping as a dictionary
    return {'damping ratio':damprat, 'natural frequency':naturalfreq, 'damped natural
    frequency':dampednaturalfreq, 'period':period, 'stiffness':stiffness, 'damping':damping}

def getfreq(heights, peakindices, x, cycles):
    #find number of peaks
    n = len(peakindices)
    #get second peak magnitude
    A1 = heights[1]
    A2 = heights[1+cycles]
    freq = cycles*2*np.pi/(x[peakindices[1+cycles]]-x[peakindices[1]]) #in rad/s
    return freq

# Path: serial.py
import serial
import matplotlib
import scipy.signal
#read serial port com4
ser = serial.Serial('COM4', 115200)

while True:

```

```

#read data from serial port
datas = []
numofdatas = 800
#wait "start" from serial port
while True:
    data = ser.readline()
    data = data.decode('utf-8')
    print(data)
    if data == 'start\r\n':
        break

for i in range(numofdatas):
    #read data from serial port if available
    data = ser.readline()
    # convert bytes to str
    data = data.decode('utf-8')
    # split data by comma
    data = data.split(',')
    # convert str to float
    data = [float(i) for i in data]
    # append data to datas
    datas.append(data)

import matplotlib.pyplot as plt
import numpy as np
#convert datas to numpy array
datas = np.array(datas)
#multiply first column by 10^3
datas[:,0] = datas[:,0]*10**-3
#remove first element of first coulumn from all elements of first column
datas[:,0] = datas[:,0]-datas[0,0]
#resample data
datas = scipy.signal.resample(datas,numofdatas)
plt.plot(datas[:,0],datas[:,1])

x=datas[:,0]
y=datas[:,1]

b=[0.00682184, 0.01364369, 0.00682184]
a=[-1,          1.7531427,  -0.78043007]

# Filter the signal
Nb = len(b)
yfilt = np.zeros(len(y));
for m in range(3,len(y)):
    yfilt[m] = b[0]*y[m];
    for i in range(1,Nb):
        yfilt[m] += a[i]*yfilt[m-i] + b[i]*y[m-i];
#join yfilt and x as 2 columns numpy array
yfilt = np.column_stack((x,yfilt))
#find peaks of yfilt with x values
peaks = scipy.signal.find_peaks(yfilt[:,1],height=0.5)
#get height dictionary from peaks
heights = peaks[1]
#get height values from heights dictionary
heights = heights['peak_heights']
peakindices = peaks[0] #wrt x

```

```

logdecresults = logdec(heights,peakindices,x,3,19)
print(logdecresults)
natfreq = getfreq(heights,peakindices,x,3)

plt.plot(x,yfilt)
#plot data
#Add title and axis names
plt.title('Filtered & Unfiltered Acceleration vs Time')
plt.xlabel('Time (s)')
plt.ylabel('Acceleration (1000*g)')
plt.show()

# Send natfreq to serial port
ser.write(str(round(natfreq,2)).encode('utf-8'))
print(natfreq)
pass

```

4.5 MATHCAD Screen Shots

$$\begin{aligned}
 m1 &:= 1 \text{ kg} & m2 &:= 0.1 \text{ kg} & reducedF &:= \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ N} \\
 k1 &:= 1 \frac{\text{N}}{\text{m}} \\
 c1 &:= 0.1 \frac{\text{N} \cdot \text{s}}{\text{m}} & M &:= \begin{bmatrix} m1 & 0 \\ 0 & m2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0.1 \end{bmatrix} \text{ kg} \\
 & & C(c2) &:= \begin{bmatrix} c1 + c2 & -c2 \\ -c2 & c2 \end{bmatrix} \\
 & & K(k2) &:= \begin{bmatrix} k1 + k2 & -k2 \\ -k2 & k2 \end{bmatrix} \\
 \\
 M2 &:= \begin{bmatrix} m1 & 0 \\ 0 & 0.00001 \text{ kg} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \cdot 10^{-5} \end{bmatrix} \text{ kg} \\
 inwM2(w, c2, k2) &:= -M2 \cdot w^2 + 1i \cdot C(c2) \cdot w + K(k2) \\
 U2(w, c2, k2) &:= inwM2(w, c2, k2)^{-1} \cdot reducedF \\
 Xsingle(w, c2, k2) &:= \left| \left(U2(w, c2, k2) \cdot e^{1i \cdot w \cdot 1 \text{ s}} \right)_0 \right| \\
 Xsingle \left(1 \text{ Hz}, 0.01 \frac{\text{N} \cdot \text{s}}{\text{m}}, 0.001 \frac{\text{N}}{\text{m}} \right) &= 10 \text{ m} \\
 \\
 inwM(w, c2, k2) &:= -M \cdot w^2 + 1i \cdot C(c2) \cdot w + K(k2) \\
 U(w, c2, k2) &:= inwM(w, c2, k2)^{-1} \cdot reducedF \\
 X1amp(w, c2, k2) &:= \left| \left(U(w, c2, k2) \cdot e^{1i \cdot w \cdot 1 \text{ s}} \right)_0 \right| \\
 X1amp \left(1 \text{ Hz}, 0.01 \frac{\text{N} \cdot \text{s}}{\text{m}}, 0.001 \frac{\text{N}}{\text{m}} \right) &= 9.083 \text{ m}
 \end{aligned}$$

Integral Minimization H2:

$$w1 := 0.6 \text{ Hz} \quad w2 := 1.4 \text{ Hz}$$

$$\text{costf}(c2, k2) := \int_{w1}^{w2} X1amp(w, c2, k2) dw$$

Guess Values

$$k2 := 0.001 \frac{N}{m} \quad c2 := 0.01 \frac{N \cdot s}{m}$$

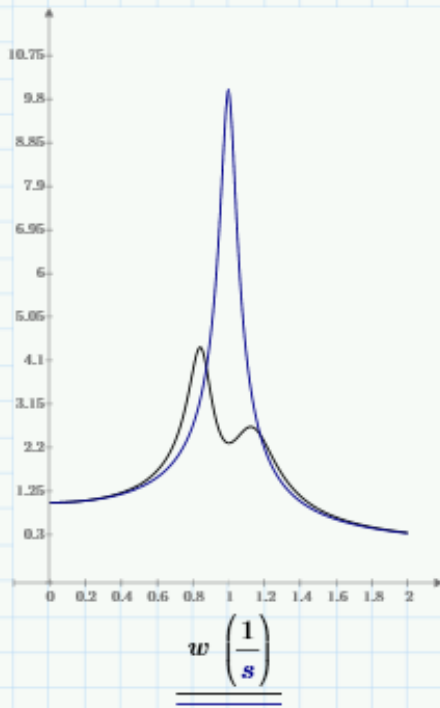
Constraints

$$k2 > 0 \quad c2 > 0$$

Solver

$$\begin{bmatrix} H2c2 \\ H2k2 \end{bmatrix} := \text{minimize}(\text{costf}, c2, k2) = \begin{bmatrix} 0.028 \frac{kg}{s} \\ 0.093 \frac{kg}{s^2} \end{bmatrix}$$

$$w := 0.001 \text{ Hz}, 0.002 \text{ Hz} \dots 2 \text{ Hz} \quad w := \vec{w} \quad 0.0293 \quad 0.0922$$



$$X1amp(w, H2c2, H2k2) \text{ (m)}$$

$$Xsingle\left(w, 0.0293 \frac{kg}{s}, 0.0922 \frac{kg}{s^2}\right) \text{ (m)}$$

$$\begin{bmatrix} 0 \\ 0.001 \\ 0.093 \end{bmatrix}$$

Peak Equate Hinf:

$$n := 0, 1 \dots 2000 \quad step := \frac{2}{2000} Hz = 0.001 \frac{1}{s}$$

$$res(c2, k2) := \overline{X1amp(wvec, c2, k2)}$$

$$wvec_n := n \cdot step =$$

$$matlol(c2, k2) := \text{augment}\left(wvec \cdot 1 \frac{s}{1 m}, res(c2, k2) \frac{1}{1 m}\right)$$

$$costf(c2, k2) := \max\left(\text{localmax}(matlol(c2, k2))^{(1)}\right)$$

$$\begin{bmatrix} 0 \\ 0.001 \\ 0.002 \\ 0.003 \\ 0.004 \\ 0.005 \\ 0.006 \\ 0.007 \\ 0.008 \\ 0.009 \\ 0.01 \\ 0.011 \\ \vdots \end{bmatrix} \frac{1}{s}$$

Guess Values

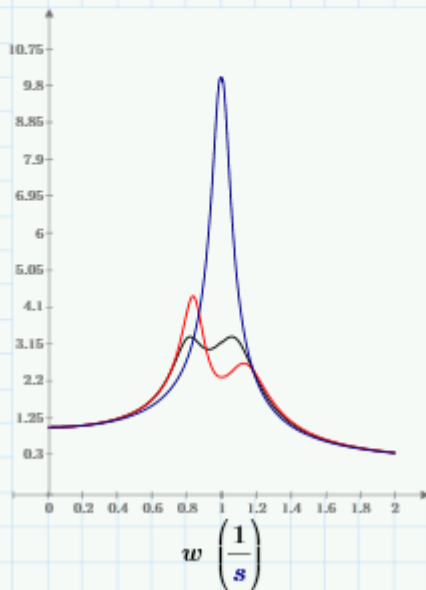
$$k2 := 0.001 \frac{N}{m} \quad c2 := 0.01 \frac{N \cdot s}{m}$$

Constraints

$$k2 > 0 \quad c2 > 0$$

Solver

$$\begin{bmatrix} Hinf c2 \\ Hinf k2 \end{bmatrix} := \text{minimize}(costf, c2, k2) = \begin{bmatrix} 0.034 \frac{kg}{s} \\ 0.079 \frac{kg}{s^2} \end{bmatrix}$$



$$\underline{X1amp(w, Hinf c2, Hinf k2) (m)}$$

$$\underline{X1amp(w, H2c2, H2k2) (m)}$$

$$\underline{Xsingle\left(w, 0.0293 \frac{kg}{s}, 0.0922 \frac{kg}{s^2}\right) (m)}$$